Plume SSD-RAM-XPU 1.1.0 — Technical Note PSX-TN-01

Nov 2, 2026 · @Russell Smithson

1. Abstract

Plume SSD-RAM-XPU 1.1.0 keeps a stack of 512 KiB Byte20 frames on the host machine's own disk. The same LCUF-A programs run either inside the storage workers or on a host engine, and the stack's SHA-256 root comes out bit-identical either way. The console runs in a web browser. Each of 1 to 16 *channel workers* owns its files in the Origin Private File System (OPFS) and holds its own copy of the Plume XPU (xpu.wasm, engine 1.3.0). A frame can therefore be advanced through one pass (BF1 then BF2) without leaving the worker that stores it; only a 160-byte reply reaches the page. A second worker exposes one eighth of the drive as byte-addressable memory, through a page cache and a double-buffered 64-byte cacheline write log modelled on SkyByte.

Release 1.1.0 makes the storage self-healing. Every frame is a pure function of (seed, frame number, generation). A frame whose bytes fail their digest is therefore recomputed from its seed, and is written back only if it reproduces the stored digest. A Scrub operation applies the same rule to the whole drive. The release also fixes five defects found in an end-to-end audit, one of which returned stale data from the SSD-RAM tier under small cache sizes.

This note specifies the on-disk format, the integrity model, the execution paths and the SSD-RAM algorithms. It reports the test evidence and the measurements taken on the build host, and maps the implementation onto the operator's design note *Architecting a 2 TiB NVMe SSD as a Unified Byte-Addressable Memory and Peer-to-Peer XPU Execution Tier* [1].

Scope. Release 1.1.0 of the single-file console `plume-ssd-ram-xpu.html` and its source tree, as built on 2 November 2026. The XPU engine, the LCUF-A language and the BF1/BF2 laws are specified in the companion note PXC-TN-01 [2] and are used here unchanged.

Status. Released. 50 Node tests and the browser end-to-end test pass. Performance figures come from a two-thread cloud build host, not from the operator's workstation (§10, §13).

2. Conventions and terminology

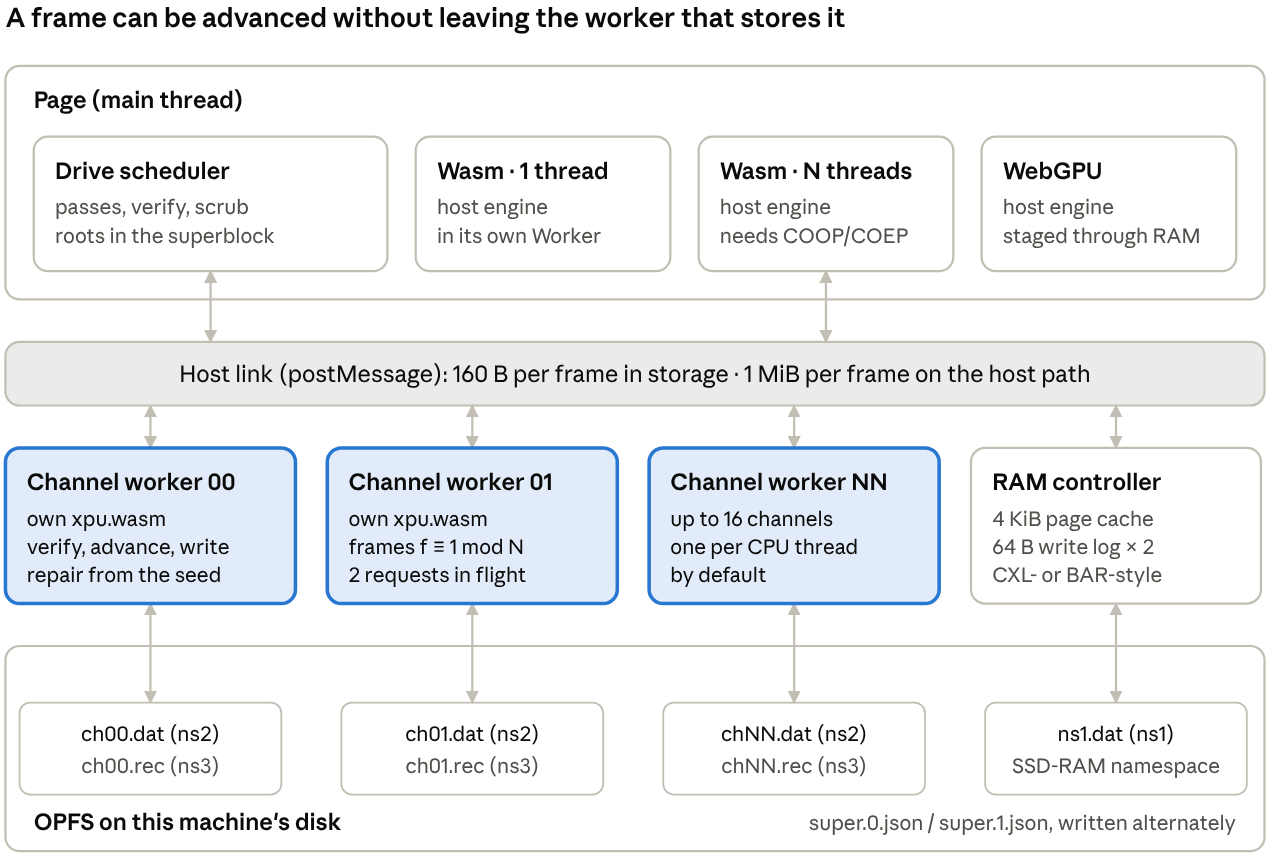
KiB, MiB, GiB and TiB are binary (2^{10}, 2^{20}, 2^{30}, 2^{40} bytes). All integers on disk are little-endian. ‖ denotes byte concatenation; u32le(x) is x as four little-endian bytes. Section numbers in square brackets refer to the References (§15).

Term	Meaning
Frame	131,072 twenty-bit words (8 lanes × 128 × 128 cells) held as u32, 524,288 bytes
F	Number of frames on the drive; frames are numbered f = 0 … F−1
N	Number of channels (1–16); frame f lives in channel c = f mod N at local index j = ⌊f / N⌋
Generation g	Number of passes a frame has undergone; generation 0 is the seed field
Pass	One application of BF1 for s_1 steps then BF2 for s_2 steps (fixed at format time)
Record	The 64-byte entry in ns3 that names a frame's generation, writing engine and digest
Root	SHA-256 commitment to all F frame digests at one generation
Channel worker	A dedicated Web Worker that owns one channel's files and one xpu.wasm instance
In-storage path	A pass executed inside the channel worker that stores the frame
Host path	A pass executed on a host engine (Wasm 1 thread, Wasm N threads, or WebGPU) after the frame is read out
RAM controller	The worker that serves ns1 as byte-addressable memory
CXL-style / BAR-style	The SSD-RAM tier with page cache and write log / with every access sent to the file uncached

Term	Meaning
Host link	The postMessage boundary between a storage worker and the page, the analogue of the PCIe root complex in [1]

The words *disk*, *flash* and *page write* describe what the console asks of the browser's file system. Whether the operating system serves a read from its page cache or from the NAND underneath is not visible to the page (§13).

3. System architecture



Plume SSD-RAM-XPU 1.1.0 · page, host link, storage workers and OPFS files

The console has three tiers. The page holds the scheduler and the host engines. The channel workers each own one channel's files and their own XPU, and the RAM controller owns ns1. Beneath them, OPFS sits on the machine's disk. Every arrow that crosses the host link is a postMessage; frame buffers move by transfer, not copy.

The layers divide the work as follows:

- **Storage runtime (`ssd.js`).** It owns the on-disk format, the integrity model, the scheduler and both worker kinds. The worker sources are built from the same functions the page uses (`frameSeed` , `seedField` , `frameDigest` , `crc32`), so the page and the workers cannot disagree on a digest.
- **Engine layer.** `xpu.c` , compiled to `xpu.wasm` and `xpu-mt.wasm` , the WGSL kernel and `engines.js` are taken unchanged from Plume XPU 1.3.0 [2].
- **Language layer.** `lcufa.js` compiles the BF1 and BF2 programs once in the page. The compiled bytecode is posted to every channel worker at start-up.
- **Page (`page.html`).** It wires these together and draws the console. It holds no state that is not also on disk, apart from rates and timings from the current session.

4. On-disk format (PLUME-SSD/1.0)

A drive is one OPFS directory, `plume-ssd-ram-xpu` . A capacity C is divided as in [1], ns1 : ns2 : ns3 + reserve = 1 : 6 : 1. The 1.1.0 changes add a record checksum and alternating superblock copies, both read compatibly from 1.0.0 drives, so the format identifier is unchanged.

4.1 Capacity plan

```
\begin{aligned}
n_1 &= \max\!\left(16\,\text{MiB},\ \left\lfloor \tfrac{C}{8\,\text{MiB}}
\right\rfloor \text{MiB}\right) \\
F &= \max\!\left(1,\ \min\!\left(2^{31}-1,\ \left\lfloor \frac{3C/4}{2 \cdot
524288 + 64} \right\rfloor\right)\right) \\
\text{reserve} &= C - n_1 - F\,(2 \cdot 524288 + 64)
\end{aligned}
```

For the note's 2 TiB drive ($C = 2^{41}$ bytes) this gives n_1 = 256 GiB, F = 1,572,768 frames occupying 1,649,267,435,520 bytes of ns2 + ns3, and a reserve of 274,877,913,088 bytes. The reserve is never allocated; it plays the role of the note's over-provisioning. The console proposes at most 90 % of the storage quota the browser grants the page.

4.2 Files

File	Namespace	Size	Owner	Contents
`ns1.dat`	ns1	n_1	RAM controller	Byte-addressable memory; zeroed (TRIM) at the start of each workload
`chNN.dat` (NN = 00–15)	ns2	J·2·524,288	channel NN	Two slots per local frame j
`chNN.rec`	ns3	J·64	channel NN	One record per local frame j
`super.0.json`, `super.1.json`	—	< 4 KiB	channel 0	Superblock copies, written alternately
`super.json`	—	< 4 KiB	—	1.0.0 superblock; read as sequence 0, never written by 1.1.0

Channel c holds $J = \lfloor (F - 1 - c)/N \rfloor + 1$ frames (0 when $c \geqslant F$). Generation g of local frame j lives at byte offset $(2j + (g \bmod 2)) \cdot 524{,}288$ of `chNN.dat`. A pass writes generation g + 1 into the other slot and only then rewrites the record, so the generation g bytes stay intact until generation g + 2 is written.

4.3 Record layout (64 bytes)

Bytes	Field	Value
0–3	magic	0x31525350 ("PSR1")
4–7	generation	u32
8–11	frame	u32 frame number f = jN + c
12	engine	0 seed, 1 in-storage Wasm, 2 host Wasm 1 thread, 3 host Wasm threads, 4 host WebGPU, 5 rebuilt from seed
13	record version	2 (1.1.0); 0 for records written by 1.0.0
14–15	reserved	0

Bytes	Field	Value
16–47	digest	frame digest (§5.1), 32 bytes
48–51	CRC	CRC-32 (IEEE 802.3, reflected, poly 0xEDB88320) of bytes 0–47; version 2 only
52–63	reserved	0

A record is *sound* when its magic and frame number are right and, for version 2, its CRC holds. A version-0 record is accepted without a CRC; any other version byte makes the record unsound.

4.4 Superblock

The superblock is JSON with the fields `format` ("PLUME-SSD/1.0"), `state` ("formatting" | "ready"), `frames`, `channels`, `seed`, `density`, `steps` [s_1, s_2], `frame_bytes`, `ns1_bytes`, `capacity_bytes`, `roots` (generation → 64-hex root), `paths` (generation → which engines wrote it), `seq`, `created`, `console`. Save k writes copy `super.(k mod 2).json` with `seq` = k. Each save writes the new bytes over the old ones, then truncates the tail, so a failure never leaves a file empty. A failed save restores `seq`, so the retry rewrites the same copy and never the last good one. Opening reads all three file names, rejects any copy that does not parse or fails range validation (frames 1–2^{31}−1, channels 1–16, steps 1–1000, density 0–100, roots 64-hex), and takes the valid copy with the highest `seq`.

5. Integrity model

Every byte the console reads from the stack is checked against a SHA-256 digest before it is used, and every frame can be recomputed from public parameters. Together these make the stack's root a reproducible function of (seed, density, F, s_1, s_2, generation). The root does not depend on the execution path, the engine, the channel count, interruptions or repairs.

5.1 Frame digest and stack root

```
\begin{aligned}
x_{f,0} &= \mathrm{seedField}\big(\mathrm{frameSeed}(\sigma, f),\
\delta\big), \qquad x_{f,g+1} = \mathrm{BF2}^{s_2}\big(\mathrm{BF1}^{s_1}
(x_{f,g})\big) \\
D_{f,g} &= \mathrm{SHA256}\big(\texttt{"PLUME-SSD-FRAME/1.0\textbackslash 0"}
\,\|\, \mathrm{u32le}(f) \,\|\, \mathrm{u32le}(g) \,\|\, \mathrm{SHA256}
(x_{f,g})\big) \\
R_g &= \mathrm{SHA256}\big(\texttt{"PLUME-SSD-STACK/1.0\textbackslash 0"}
\,\|\, \mathrm{u32le}(F) \,\|\, \mathrm{u32le}(g) \,\|\, \mathrm{u32le}(s_1)
\,\|\, \mathrm{u32le}(s_2) \,\|\, D_{0,g} \,\|\, \cdots \,\|\, D_{F-1,g}\big)
\end{aligned}
```

σ is the 32-bit seed and δ the plane-0 density in percent. frameSeed is a 32-bit avalanche mix of σ and f + 1; seedField is a xorshift32 generator that fills each word with a random even 20-bit value and sets bit 0 with probability δ %. The frame number and generation are bound into D, so a frame cannot be swapped with another frame or replayed at another generation without detection. Hashing the frame once and then hashing a fixed-size header avoids copying 512 KiB per digest.

5.2 Invariants

1. **I1 (record truth).** For every sound record (f, g, D), the bytes in slot g mod 2 equal x(f, g), or the record is detected as unsound or mismatched on its next read.
2. **I2 (single-step advance).** A pass reads only frames whose record generation equals the drive's lowest generation g, and writes generation g + 1. The worker re-checks the expected generation before any read or repair.
3. **I3 (generation cap).** No sound record names a generation above max(roots) + 1. A larger value can only come from damage, so such a record counts as unsound.
4. **I4 (root at uniform generation).** A root is computed only when every record names the same generation. It is stored in the superblock and compared with any root previously recorded for that generation.

5.3 Path independence

The in-storage XPU (xpu.wasm inside the channel worker), the host Wasm engines (xpu.wasm, xpu-mt.wasm) and the WGSL kernel are required by the 1.3.0 engine suite to produce bit-identical frames for the same program, input and step count [2]. The digest depends only on (f, g, x), and x(f, g) is fixed by I2 and the seed. R_g is therefore the same

for any assignment of frames to paths, engines or channels. Release 1.1.0 tests this directly: in-storage on three channels, host Wasm on two, cooperative, and stop-then-resume on mixed paths all give identical roots (§9).

5.4 Verified reads and self-repair

Every `compute` (in-storage) and `read` (host) request runs `readChecked`:

1. Read the record and require that it is sound and names the expected generation g.
2. Read slot g mod 2 and compute D′. If D′ = D, use the bytes.
3. Otherwise recompute x(f, g) from the seed inside the worker (g passes on its own XPU) and compute its digest. If it equals D, write it to the slot, flush, and continue; the reply carries `repaired: 1`.
4. If the recomputation does not reproduce D, the record itself is wrong. The request fails with a pointer to Scrub, and nothing is written.

The cost of a repair is g passes on one thread, paid only for damaged frames.

5.5 Scrub

Scrub reads every frame on every channel in parallel, 16 frames per request, and classifies each one:

- **good**: sound record and matching digest;
- **repaired**: the bytes were wrong, and a recomputation reproduced the record;
- **rebuilt**: the record was unsound, named a generation above the cap, or disagreed with the recomputation. The frame is recomputed to the record's generation, or to the drive's most common generation when the record is unusable, and a fresh record is written with engine code 5.

After a scrub, frames may sit one or more generations behind the rest. The next Run advances only the frames at the lowest generation, repeating until all agree (I2); then the root is derived and compared with the stored one.

6. Execution paths

A pass can run on the in-storage path, the host path, or both at once. The in-storage path moves 160 bytes per frame across the host link and the host path moves 1 MiB, while the results are identical. Every path is resumable: a stopped or failed pass leaves each frame at generation g or g + 1, and the next Run advances only those still at g.

6.1 In-storage path

Each channel worker processes its own list of frames at generation g. The page keeps two `compute` requests in flight per worker, so the worker never waits for a message between frames. Within the worker, requests are serialised by a promise chain, so no two requests touch the same files at once. For each frame the worker:

1. runs `readChecked` (§5.4);
2. configures its XPU for BF1 ($Z/2^{20}$), runs s_1 steps, reconfigures for BF2 ($Z/5^8$) and runs s_2 steps;
3. writes the result to slot (g + 1) mod 2, computes D(f, g + 1) and writes the version-2 record;
4. flushes both files every 16 frames and at the end of the pass.

The reply holds the frame number, the generation, the 32-byte digest and the timings: 160 bytes as accounted by the console. A 16 KiB scope thumbnail is attached at most every 250 ms.

6.2 Host path

The host path runs q *lanes* ($1 \leqslant q \leqslant 64$, the queue depth). Each lane repeatedly:

1. takes the next frame, round-robin across the channels assigned to the host;
2. issues `read`; the worker verifies the frame and transfers its 512 KiB buffer to the page without copying;
3. runs BF1 then BF2 on the selected host engine. Engines serialise their own runs, so lanes overlap I/O with compute but not compute with compute;
4. issues `write`, transferring the result back. The worker range-checks every word ($\leqslant$ 0xFFFFF), refuses a generation other than g + 1, writes the slot, digest and record.

The page measures the read latency L (request to reply) and the read throughput T, and reports the requests in flight by Little's law:

```
Q = T \times L
```

When Q falls well below q, the host engine, not the disk, sets the pace, and the console says so. On the build host Q was 0.13–0.15 against q = 8 (§10).

6.3 Cooperative path

The first k channels serve the host path and the remaining N − k compute in storage, concurrently. Let h be the host path's rate (frames/s) and s the rate of one in-storage channel. Each side finishes at the same time when

```
\frac{kF/N}{h} = \frac{(N-k)F/N}{(N-k)\,s} \quad\Longrightarrow\quad k = \frac{h}{s}
```

The console therefore suggests k = round(h / s), clamped to 1 … N − 1, from the rates of the last runs. The host path receives as many channels as it is worth in-storage workers.

6.4 Failure semantics

The storage loops and host lanes are awaited with `allSettled`. The first failure sets the drive's stop flag, every other loop finishes its current frame and stops, and the pass rethrows the first error only after all have ended. No write can therefore land after a pass has returned (defect D3, §12). Stop behaves the same way without an error. A message from a worker without a request id marks that worker dead: its pending requests are rejected at once and later calls fail immediately instead of hanging.

7. SSD-RAM tier (plane I)

The RAM controller serves ns1 as a word-addressable memory. In CXL-style mode it puts a 4 KiB page cache and a double-buffered 64-byte cacheline write log in front of the file; in BAR-style mode every access goes to the file. With the defaults, CXL-style was 13.3× faster per operation with compaction included and wrote 19.5× fewer pages (§10), with every load and every read-back correct.

7.1 Workload

Each operation picks a 20-bit word index w = ⌊W · u^{1/α}⌋, where W is the number of words in ns1, u is uniform in (0, 1) and α ∈ (0, 1] sets the locality, after the Apex-Map benchmark cited in [1]. α = 1 is uniform, and small α concentrates accesses near the start of the namespace. Each operation loads the word; with probability equal to the store percentage it then stores T(x) = x + (x² ∨ 5) mod 2^{20}, the BF1 Klimov–Shamir map. The run starts from a zeroed namespace (truncate to 0, then to n_1, the analogue of TRIM). A shadow map holds the value last stored at every touched address, and every load is compared with it. After the run both log halves are compacted, the file is flushed, and every touched word is read back from the file directly, bypassing the cache.

7.2 Structures (CXL-style)

Structure	Organisation	Default size
Data cache	P pages of 4 KiB, clock (second-chance) eviction, page → slot map	56 MiB (14,336 pages)
Write log	Two halves (active, frozen), each L lines of 64 B, append-only	8 MiB (2 × 65,536 lines)
Index	Per half: page → (line offset → log slot), after SkyByte's two-level hash	Map entries

The defaults follow SkyByte's 1 : 7 split of device DRAM between log and cache [3], at 1/8 of a 512 MiB slice.

7.3 Operations

- **load(a)**: look up line ⌊a/64⌋ of page ⌊a/4096⌋ in the active log, then the frozen log, then the cache. On a miss, read the page from the file into a cache slot chosen by clock, then lay every logged line of that page (frozen first, then active) over it.
- **store(a, v)**: if the line is in the active log, write the word there. Otherwise take the line's current 64 bytes, from the frozen log, the cache, or a cache fill. Copy them into a new active slot, and only then insert the slot into the index. Write the word, and update the cached page if present. If the active half is full, first compact the frozen half, then swap the halves.
- **compact(L)**: for each page with lines in half L, take the cached copy if present (it already holds every logged line); otherwise read the page and apply L's lines. Write the page once, then clear L.

7.4 Coherence invariant

At every instant the value of any word equals, in order of precedence, its line in the active log, else its line in the frozen log, else its cached page, else the file. Every cached page already includes every logged line of that page. Stores keep this by updating the cached copy; fills keep it by applying the logs after the read. Release 1.0.0 broke it for one case (defect D1, §12): a new slot was indexed before its bytes were copied, so a cache fill in between applied the slot's stale bytes to the page.

7.5 BAR-style baseline

Every load reads 4 bytes at the word's address and every store writes 4 bytes, with no cache or log. Each store counts as one device page write, as a NAND page program would on a real SSD. Because BAR-style is about 40× slower per operation, 1.1.0 runs it on the first 20,000 operations of the same sequence and compares per-operation figures.

7.6 Timing

The loop time excludes compaction, which SkyByte runs in the background, and the time spent yielding for a Stop request. The console reports both µs per operation for the loop and µs per operation including compaction and the final persist. Timings are aggregated over the run, because browser clocks may be coarsened to 100 µs outside cross-origin isolation.

8. Failure model and crash consistency

No single fault in the stack loses data that can be recomputed, and every fault is either repaired automatically or reported with the action that clears it. The model assumes that the file system may lose or tear any unflushed write, that it may silently corrupt stored bytes, and that the tab may be closed at any instant. The SHA-256 and CRC-32 checks are assumed sound; deliberate forgery of digests is out of scope.

Fault	Where it can occur	Detection	Outcome in 1.1.0
Frame bytes damaged at rest	any slot	digest mismatch on read (§5.4)	recomputed from the seed, checked against the record, rewritten; pass continues (`repaired`)
Crash after the record write, before the data reached disk	in-storage or host write, ⩽ 16 frames since the last flush	digest mismatch on next read	as above: the record names generation g + 1, which is recomputable

Fault	Where it can occur	Detection	Outcome in 1.1.0
Crash after the data write, before the record write	any write	none needed	record still names g; slot g is intact (other slot); the frame is redone by the next Run
Record digest or fields damaged	ns3	CRC-32 mismatch, wrong magic or frame number	pass refuses before advancing anything ("run Scrub"); Scrub rebuilds the frame and record
Generation field damaged into a large value	ns3	CRC-32, and the cap $g \leqslant \max(\text{roots}) + 1$	treated as an unsound record; Scrub rebuilds it in milliseconds instead of running 2^{24} passes (defect D2)
Record lost entirely	ns3	unsound record	Scrub rebuilds at the drive's most common generation; Run brings it level
Superblock copy torn or empty	`super.k.json`	JSON parse or range validation	the other copy, one sequence older, is used
Superblock save fails (quota)	channel 0	rejected `putText`	`seq` restored; the retry rewrites the same copy; the last good copy is untouched (defect D5)
Host engine fails mid-pass	host lanes	rejected `run`	all lanes stop, nothing is written after the error, the next Run resumes (defect D3)

Fault	Where it can occur	Detection	Outcome in 1.1.0
Storage worker dies	any	message without a request id	pending and later requests fail at once; reload reopens the drive
Drive open in another tab	OPFS access-handle lock	open fails with a lock error	reported as such; Run, Verify and Scrub stay disabled
Format interrupted	any	superblock `state: formatting`	Resume format keeps frames whose generation-0 record is sound and writes the rest
Quota exhausted while writing	any	write accepts no bytes, or a QuotaExceeded error	the request fails with the offset; workers and locks are released (defect D4)

Durability is bounded by the flush interval: up to 16 frames per channel may need recomputation after a power loss, plus the frames in flight. Because every frame is recomputable, the bound limits time to recover, not data loss. The SSD-RAM tier has no persistence contract across runs: each workload starts with TRIM, and its read-back runs after an explicit flush.

9. Verification and test evidence

Release 1.1.0 passes all 50 assertions of the Node suite and the browser end-to-end test. Before release, an independent review that had not seen the implementation reasoning found five defects; each now has a regression test (§12).

9.1 Node suite (`test/ssd.test.mjs`)

The Node suite runs the real channel and RAM-controller worker sources in `worker_threads`, over a plain-file backend with the same interface as OPFS synchronous access handles.

Group	Assertions	What is established
Capacity plan	2	exact frame count; 2 TiB splits 1 : 6 : 1 as in [1]
Format	3	generation-0 root identical on 3 and 2 channels; stored digest equals the seed field's
Paths	5	in-storage (3 channels) = host Wasm (2 channels) = cooperative; host-link bytes 0 vs 2 frames per frame; superblock holds the root
Reference	1	after two passes, the JavaScript reference interpreter reproduces the stored digest of frame 4
Verify	1	five sampled frames recomputed on a separate worker engine match; root re-derived
Stop and resume	4	stop after 6 of 12 frames; resume on another path skips them; mixed-path root = uninterrupted root on 4 channels; reopen reads records
Format resume	2	stopped format leaves state "formatting"; resume finishes with the uninterrupted root
Damage, self-repair, scrub	9	flipped data byte repaired in an in-storage pass and in a host pass with roots unchanged; damaged record refused before any advance; scrub reports 7 good / 1 repaired / 2 rebuilt and re-derives the root; torn newest superblock falls back one sequence; impossible superblock refused
Review findings	8	gen field $2^{24}+1$ counts as damaged and scrub fixes it in 49 ms; no frame writes after a failed host pass; four small-cache SSD-RAM settings with 0 stale reads through 51–472 log swaps
1.0.0 compatibility	1	version-0 records and `super.json` open; the next save starts the alternating copies
Host write check	2	a 23-bit word from a host engine is refused and nothing advances

Group	Assertions	What is established
SSD-RAM	8	CXL-style high locality and uniform, BAR-style: every load and read-back correct; coalescing 29,818 stores → 2,404 page writes; Stop honoured after 203,776 operations; parameters range-checked
Model	4	Little's law examples from [1] (750 and 35); (s+1)/2 and α* formulas; T on $Z/2^{20}$

9.2 Browser end-to-end test (`tools/smoke.cjs`)

The end-to-end test runs headless Chromium with software WebGPU against the real OPFS, behind the cross-origin-isolated server. On a 0.25 GiB drive (191 frames, 4 channels, seed 7, density 35 %, $s_1 = 4$, $s_2 = 2$) it:

1. starts a format, stops it, and resumes it;
2. formats, runs an in-storage pass, re-formats, and runs a host pass on Wasm with 2 threads, requiring both roots to be equal;
3. runs two cooperative passes and a scrub (191 good, 0 repaired, 0 rebuilt, root re-derived);
4. verifies three frames on WebGPU;
5. runs the SSD-RAM comparison and the medium benchmark;
6. reloads the page and requires the drive to reopen at generation 3, then erases it.

Any page error fails the test.

9.3 Reference roots

These values allow an independent implementation to be checked. Parameters: F = 191, seed 7, density 35 %, $s_1 = 4$, $s_2 = 2$. The root depends on F, so these hold for a 0.25 GiB drive on any channel count.

Generation	Root R_g
0	`09f9dd79e27416c48f730603fc9d7a7302f542b2762757a1a4ff7d0b08b88a86`

Generation	Root R_g
1	`98933969c0114809c3f2d21dd2925deba462208c3a1dfc13eccf8983ebe3c687`
2	`a128b4eb06bbf06bb0d00c15426570767ab38b00cc1f549df1c629b27f2c9002`
3	`331a1d322bd80f284a3b2c2f3559872e6b7ba0459564afccc7d319aeb1131a4d`

Generation 1 was produced both by the in-storage path and by the host path on a separately formatted drive. Generations 2 and 3 came from cooperative passes; the 1.0.0 and 1.1.0 test runs gave the same values.

10. Measurements

On the build host the in-storage path moved 6,554× less data across the host link than the host path, at comparable frame rates. The CXL-style SSD-RAM tier cost 13.3× less per operation than uncached access. All figures come from the 1.1.0 browser test runs on a two-thread cloud container (headless Chromium under Playwright 1.63, software WebGPU, 0.25 GiB drive, 191 frames, 4 channels, $s_1 = 4$, $s_2 = 2$). They show relative behaviour, not hardware capability (§13).

10.1 Stack passes

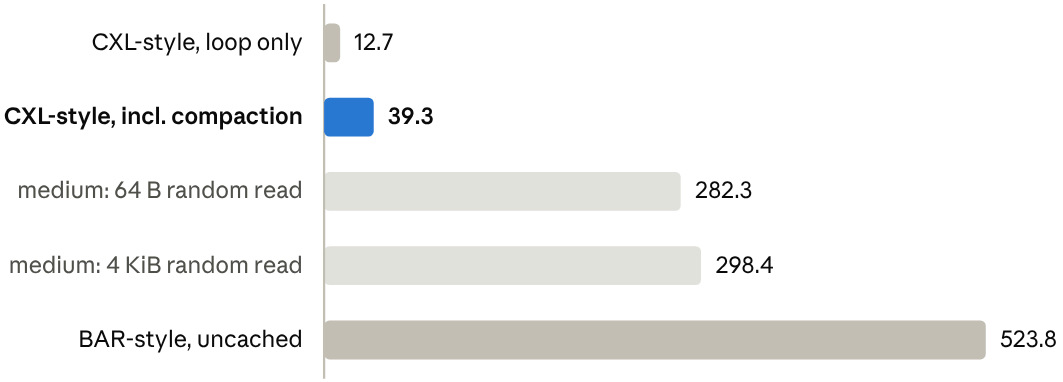
Path	Frames / s	Host-link traffic per pass	Notes
In-storage, 4 channel workers	32 (8.0 per channel)	29.8 KiB	4 workers share 2 CPU threads
Host, Wasm · 2 threads, q = 8	33.5	191.0 MiB	L = 3.81 ms, T = 33.5/s, Q = T × L = 0.13
Cooperative, 3 host + 1 storage channel	40.5	144.0 MiB + 7.3 KiB	host 30.5 frames/s; storage 10.0 frames/s per channel

Formatting wrote generation 0 at 32.5 MiB/s, including seed generation and two SHA-256 digests per frame. Because Q stayed near 0.13 with eight requests allowed, the host engine, not storage, bounded the host path. On a machine with more cores than channels, the in-storage path scales with the number of channels while the host path is bound to one engine.

10.2 SSD-RAM tier

CXL-style SSD-RAM costs 39 µs per operation with compaction, 13× less than BAR-style

µs per operation on ns1 (load + 50 % stores, α 0.02), build host, headless Chromium; shorter is faster



BAR-style ÷ CXL-style incl. compaction = 13.3× · lighter bars: the medium's own random-read latency

tools/smoke.cjs, 1.1.0 release run, 2 November 2026 · 100,000 CXL-style and 20,000 BAR-style operations, 4,096-read medium benchmark

The CXL-style tier served 96.2 % of loads from the log or cache, with 3,824 page misses in 100,000 operations. It wrote 2,564 pages for 49,978 stores, a 19.5× page-write saving. Log compaction tripled its cost per operation, from 12.7 to 39.3 µs, because a single worker pays in the foreground what SkyByte does in the background. A 64-byte read of the medium cost almost as much as a 4 KiB read, so BAR-style pays the browser's per-call overhead on every access. Every one of the 120,000 loads and 10,780 read-backs was correct.

11. Correspondence with the SSD-RAM-XPU design note

The console implements each of the note's four planes with the mechanism a browser offers, and shows the note's hardware figures only in a labelled model panel. Nothing in this release reaches CXL, the NVMe controller or raw logical block addresses (LBAs).

Note [1]	Plume SSD-RAM-XPU 1.1.0	Status
2 TiB drive in three NVMe namespaces, 256 : 1,536 : 256 GiB	One OPFS directory split 1 : 6 : 1 by `planCapacity`; 2 TiB gives the same split	implemented, scaled to the browser quota
Plane I: CXL.mem Type-3 SSD-RAM with SkyByte write log and data cache	RAM controller: 64 B cacheline write log (double-buffered, two-level index) + 4 KiB page cache, 1 : 7	implemented in software; no CPU cacheability, no CXL flits
Plane I: PCIe-BAR uncached baseline	BAR-style mode: every access to the file	implemented as a comparison
Plane I: SkyByte-Delay context switch on long misses	not implemented; misses block the controller worker	modelled only
Plane II: BaM / GPUDirect XPU-initiated storage, Little's-law queue depth	Host path with q lanes, measured L, T and Q; frames staged through RAM	implemented without P2P (browsers have no disk-to-GPU path)
Plane III: near- / in-storage processing (SmartSSD, HILOS)	Channel workers run xpu.wasm on their own frames; 160 B per frame to the page	implemented; the "storage" is a worker, not a drive controller
HILOS host-traffic ratio (s + 1)/2	Model panel; measured analogue 1 MiB / 160 B = 6,554× per frame	modelled + measured analogue
HILOS cooperative split α* = 2B_PCI / (B_SSD + B_PCI)	Model panel; cooperative mode uses k = round(h/s) from measured rates (§6.3)	modelled + implemented analogue
Plane IV: raw-LBA I/O, exact-size pinned buffers, fused overflow check	Fixed-offset flat files, exact 512 KiB frame buffers, fused 20-bit range check on host writes	implemented analogue
Endurance (write-amplification reduction, delayed writeback)	Page-write saving reported per SSD-RAM run; frames written once per generation	measured analogue

Note [1]	Plume SSD-RAM-XPU 1.1.0	Status
Verification gates 1–4 (gdscheck, nvm-block-bench, STREAM, raw LBA)	Node suite, browser end-to-end test, Verify, Scrub (§9)	replaced by functional and integrity tests

The model panel reproduces the note's worked examples exactly. For Little's law it gives 16 GB/s × 3 µs / 64 B = 750 and 32 GB/s × 70 ns / 64 B = 35. For a TLC drive under load (324 µs) on a 26 GB/s link with 4 KiB requests it computes Q = 2,057, matching the note. The note quotes 16,524 for 512 B requests; computing 26 GB/s exactly gives 16,453, because the note rounds the request rate to 51 M/s. The note's latency, bandwidth and cost figures are reproduced as given, with their sources, and were not independently checked for this note.

12. The 1.1.0 audit

The audit of 1.0.0 found five defects. Three were reproduced with scripts before they were fixed. The most serious, D1, returned wrong data from the SSD-RAM tier; it went unnoticed because the 1.0.0 tests used cache sizes large enough never to reach it. The 1.0.0 frame stack never returned wrong data: its digests caught every corruption, but several faults could only be cleared by re-formatting.

Id	Severity	Defect in 1.0.0	Root cause	Fix in 1.1.0	Regression test
D1	high: wrong data	SSD-RAM loads returned stale words after a log swap (341 wrong loads in 60,000 at 16 pages / 64 lines / α 0.2)	`store` indexed a new log slot before copying the line into it; a cache fill in between applied the slot's stale bytes, then copied them back	the slot is indexed after the copy (§7.4)	4 small-cache settings, 0 stale reads
D2	high: hang	one flipped bit in a record's generation field hung Scrub (2^{24} passes in one worker) and made Run loop	records carried no checksum; the scrub cap was derived from the same damaged scan	CRC-32 in record version 2; generation cap max(roots) + 1; catch-up loop limited to 1,000 passes	generation $2^{24}+1$ counts as damaged; scrub in 49 ms

Id	Severity	Defect in 1.0.0	Root cause	Fix in 1.1.0	Regression test
D3	medium: late writes	after a host engine failed, other lanes kept writing frames after the pass had returned	`Promise.all` rejects on the first failure without waiting for the rest	`allSettled`; the first failure stops every lane	no generation changes after the failure
D4	medium: locks	a failed format, resume or SSD-RAM start left workers holding OPFS locks, so Erase and Format failed until reload	failure paths dropped references without closing workers	workers closed on every failure path	covered by the format and resume tests
D5	medium: data loss	two failed superblock saves in a row could empty both copies	the sequence number advanced on failure; the save truncated before writing	`seq` restored on failure; write over, then cut the tail	torn newest copy falls back one sequence

The audit also added self-repair on read, Scrub, the Passes box, Stop for Verify and SSD-RAM runs, superblock validation, reporting of a drive held by another tab, and the µs-per-operation column that includes compaction. That column lowered the reported CXL-style speed-up from 46× to 13.3× on the build host: the 1.0.0 figure left out SkyByte's background compaction, which a single-threaded worker has to pay in the foreground.

13. Limitations and threats to validity

The figures in this note show that the design works and that its parts compare as expected. They do not measure SSD hardware.

- **OS page cache.** OPFS sits on the operating system's file system. Reads of recently written data are likely served from RAM, so the measured "disk" latencies are lower bounds for flash and upper bounds for DRAM. No experiment here forces a cold read.
- **Build host.** All measurements come from a two-thread cloud container running headless Chromium with SwiftShader (software WebGPU). The operator's workstation (8 threads, AMD GCN-5 GPU) has not run 1.1.0. Its absolute rates, and the host-versus-storage balance, will differ.
- **OPFS overheads.** A synchronous access handle crosses into the browser's storage process. On the build host this made a 64-byte read cost almost as much as a 4 KiB read (282 vs 298 µs), which flattens the BAR-style penalty compared with real PCIe

MMIO.

- **Durability.** `flush()` is the strongest guarantee the browser offers; whether it reaches stable media depends on the OS and the drive. Recovery relies on recomputation, which is bounded by the flush interval (§8).
- **Recompute cost.** Repairing a frame at generation g costs g passes on one thread. For stacks advanced thousands of generations, a scrub of a badly damaged drive can take as long as the original passes.
- **Single writer.** Only one tab may hold the drive; a second tab is refused rather than coordinated.
- **No adversary.** Digests protect against accidental damage. Anyone with access to the browser profile can rewrite frames, records and superblocks together.
- **Model figures.** The hardware latencies, bandwidths, costs and speed-ups in the model panel are taken from the operator's note and its sources, and were not re-verified for this note.

14. Reproducibility

The release is reproducible from `plume-ssd-ram-xpu-1.1.0.zip` with clang 18 (wasm32), Python 3, Node 22 and Playwright 1.63. The two Wasm modules are byte-identical to Plume XPU 1.3.0.

Artefact	Size	SHA-256
`dist/plume-ssd-ram-xpu.html` (single-file console)	167,946 B	`e9c0d9a458f085d880181fc5ee549a840af659d327dd381d4ef1aefb11ef5d32`
`web/xpu.wasm` (1 thread; also the in-storage XPU)	11,814 B	`41ba8f680b14928d6b37957bd1fb7856889f6b869dee012470676f02ccd00795`
`web/xpu-mt.wasm` (shared-memory threads)	11,780 B	`c67ea14ee5f5b970759e08e6fa7bbf63c9b55653047dbaf802edfe29dc30bfb6`
`plume-ssd-ram-xpu-1.1.0.zip` (source release)	183,871 B	`bf3f3dd6e499507ec6b384eca521c3394c7ccc914e765d8db5f1048cd0ae5b90`

```
./build.sh                # wasm32 build, bundle, then the browser end-to-
end test
node test/ssd.test.mjs    # 50 assertions on Node worker_threads + plain
files
node tools/smoke.cjs      # headless Chromium against the real OPFS
node serve.mjs            # http://127.0.0.1:8789/ with COOP/COEP, so Wasm
threads work
```

On Windows, `Start SSD-RAM-XPU Console.cmd` starts `serve.mjs` on the portable Node shipped with ByteFoundry 0.99.2 and opens the browser. Wasm threads require the cross-origin isolation that `serve.mjs` provides. The published single-file page runs the in-storage path, the 1-thread host engine and WebGPU wherever its host grants OPFS and `crypto.subtle`.

The source is 668 lines in `src/ssd.js`, 655 in `src/page.html`, 175 in `test/ssd.test.mjs` and 49 in `tools/smoke.cjs`, plus the unchanged 1.3.0 engine. It is licensed GPL-3.0-only, Copyright (C) 2026 RUSSELL PHILIP SMITHSON; see LICENSE, COPYRIGHT and NOTICE.

15. References

1. Operator's design note, *Architecting a 2 TiB Non-Volatile Memory Express Solid-State Drive as a Unified Byte-Addressable Memory and Peer-to-Peer XPU Execution Tier* (supplied 2 November 2026), and the works it cites.
2. *Plume XPU Console 1.2.0 and 1.3.0 — Technical Note PXC-TN-01*, 2 November 2026: the XPU engine, LCUF-A/0.3, the BF1 and BF2 laws, and engine equivalence. Claude Docs
3. H. Zhang et al., "SkyByte: Architecting an Efficient Memory-Semantic CXL-based SSD with OS and Hardware Co-design," HPCA 2025. arXiv:2501.10682
4. M. Kwon et al., "From Block to Byte: Transforming PCIe SSDs with CXL Memory Protocol and Instruction Annotation," IEEE Micro 2025. arXiv:2506.15613
5. Z. Qureshi et al., "GPU-Initiated On-Demand High-Throughput Storage Access in the BaM System Architecture," ASPLOS 2023. arXiv:2203.04910
6. H. Jang et al., "A Cost-Effective Near-Storage Processing Solution for Offline Inference of Long-Context LLMs" (HILOS), ASPLOS 2026. arXiv:2502.09921
7. Y.-C. Liaw and S.-H. Chen, "MemAscend: System Memory Optimization for SSD-Offloaded LLM Fine-Tuning," 2026. arXiv:2505.23254
8. A. Klimov and A. Shamir, "A New Class of Invertible Mappings," CHES 2002, LNCS 2523, pp. 470–483. doi:10.1007/3-540-36400-5_34

9. WHATWG, *File System Standard* (Origin Private File System, FileSystemSyncAccessHandle). fs.spec.whatwg.org
10. NIST, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015.
11. ISO/IEC 8802-3 (IEEE 802.3) frame check sequence: the CRC-32 polynomial 0x04C11DB7, reflected form 0xEDB88320.
12. Plume SSD-RAM-XPU 1.1.0 source: `src/ssd.js`, `src/page.html`, `test/ssd.test.mjs`, `tools/smoke.cjs`, `CHANGELOG.md`, GPL-3.0-only.

References 3–7 are cited as listed in [1]; their figures appear here only in §11's model comparison.